Agent Deployment into Coupled Systems

A Working Framework

John P. Alioto

Version 1.0
April 2026

Preface

This is a working framework for analyzing the deployment of AI agents into coupled operational systems. It is intended for engineers, architects, operators, and governance practitioners who have already encountered the dysfunction this framework names and need vocabulary, structure, and predictions they can use in their own work.

The framework is structural rather than prescriptive. It identifies the primitive that AI agents are built around, the composition path from primitive to autonomous system, the deployment error that produces the central failure mode, and the dynamics that govern how that failure propagates through coupled engineered systems. It compiles predictions about deployment tempo and attribution. It stops short of architectural prescription, which depends on conditions specific to organizations and remains the work of subsequent analysis.

The framework is dense by design. It uses formal notation where formal notation clarifies, prose where prose clarifies, and aphoristic compression where compression aids retention. It does not survey the AI safety literature. It does not hedge claims to satisfy peer-review conventions. It engages the strongest counterposition — the deployment-defender argument that real-world stress testing produces information lab-only testing cannot supply — directly, and stops short of committing to which deployment regime current conditions are in, because the regime is empirical and partial observability cuts both ways.

Readers familiar with the practitioner-facing introduction "Diagnosing Hidden Control Edges in Agent Systems" will recognize this framework as the structural grounding for that piece. Readers arriving here directly will find the diagnostic framing developed alongside the structural argument. Both entry points lead to the same conclusion: agent deployment is revealing structural pathologies in systems we have already built, and the first job is to name the shape of what the diagnostic reveals.

The framework's central claim is that we are composing agents around an irreducible primitive, industrializing that composition through frameworks and platforms, and deploying the resulting autonomous composites into operational closures whose structure is only partially known. The likely result is a large collision between partially understood agents and partially understood systems that many organizations mistakenly believe they understand.

The notation table in section 2 establishes conventions used throughout. Section 1 states the thesis. Subsequent sections develop the argument from the primitive forward. Readers may read sequentially or use the section headings as a reference index. The framework is intended to be cited

section by section as practitioners apply it to specific organizational decisions.

The framework is a working document. It will be revised as additional incidents, predictions, and analytical refinements emerge.

Contents

Preface **1**

1 Thesis **3**

2 Notation and Terminology **3**

3 The Primitive **6**

4 Composition **6**

5 Industrialized Composition **8**

6 Internal Non-Interference Defect **8**

7 Operational Stratification **9**

8 Understanding and False Closure **10**

9 Human Onboarding Counterfactual **11**

10 Assumed Shared World Model **11**

11 Consequence-Coupled Inhibition **12**

12 Interface Coupling Is Not Structural Calibration **13**

13 Latent Susceptibility **14**

14 Trajectory Prediction Burden **14**

15 Corollary: Action-State Non-Recoverability **15**

16 Cascade Gain **16**

17 Response-Loop Amplification **18**

18 Partial Operational Graph **18**

19 Endogenous Actor Response **20**

20 Scale-Recursive Coupling **21**

21 Situated Prediction **21**

22 False Closure and Collision **22**

23 Tempo Uncertainty **22**

24 Deployment Tempo, Stress Testing, and Learning **23**

25 Viability Under Partial Observability **25**

26 Formal Boundary **25**

27 Closing Note: Collision and Regime Change **26**

References **26**

---

1. Thesis

An LLM core is not an agent. It is an irreducible **logit-transition primitive**. An AI agent is an engineered composite built around that primitive through decoding, memory, tools, context construction, state update, validation, authority, and orchestration.

The deployment problem begins when the agent is connected to coupled operational systems whose dependency structure is only partially known. The agent becomes operationally coupled through interfaces, tools, credentials, documents, APIs, workflows, and records, but it is not thereby structurally calibrated to the system's **actual operational graph**.

The central failure mode is **false closure**: treating partial maps, local coherence, documented workflows, demos, tests, dashboards, and process descriptions as if they constituted counterfactually stable understanding of the system.

This framework does not claim that agent deployment is inherently destructive. Agent deployment is a stressor. It may damage systems, reveal latent structure, force better instrumentation, or produce the learning required to build larger and more viable systems later. From inside the system, the sign cannot be known in advance.

The claim is narrower:

> We are composing agents around an irreducible primitive, industrializing that composition through frameworks and platforms, and deploying the resulting autonomous composites into operational closures whose structure is only partially known.

The likely result is a large collision between partially understood agents and partially understood systems that many organizations mistakenly believe they understand.

2. Notation and Terminology

This framework uses the following conventions. The transformer-pipeline notation aligns with standard usage in the autoregressive language modeling literature.

Model primitive (the autoregressive pipeline). The primitive is not a single mapping. It is the standard transformer pipeline that takes context to a token:

$$c_t \xrightarrow{f_\theta} h_t \xrightarrow{G_\theta} z_t \xrightarrow{K_\varphi} x_{t+1}$$

Symbol	Meaning
θ	Fixed model weights.
c_t	The accumulated context at time t. The notation $c_{\leq t}$ is used only when accumulation needs emphasis.
f_θ	The model's mapping from context to hidden state: $h_t = f_\theta(c_t)$.
h_t	The model's conditioned hidden state at time t.
G_θ	The projection from hidden state to logits: $z_t = G_\theta(h_t)$.
z_t	The logit field over vocabulary set V at time t. The vocabulary size is $\|V\|$.
K_φ	The sampling/decoding policy that collapses logits to a token (temperature, top-k, etc.): $x_{t+1} = K_\varphi(z_t)$.
x_t	The token at position t in the autoregressive sequence. The model output x_{t+1} becomes part of the next input via c_{t+1}.

Composition layer (around the primitive).

Symbol	Meaning
C	The composite control layer around the primitive: parsers, validators, policy checks, orchestration, authority boundaries, and related machinery.
U	The context/state update operator that incorporates decoded output, tool observations, memory, and environment feedback into later context.
a_t	An external action or tool call produced after parsing the model's output.
o_t	An observation available to the agent, harness, operator, or organization.
η_t	Agent harness state: step counter, planner state, retry state, and runtime bookkeeping.

System and operational layer.

Symbol	Meaning
S_t	The true environment/system state at time t. It is generally only partially observed.
Ω_t	The full joint latent state at time t. Introduced formally in section 15.
μ_t	Retrieval/context-construction state at time t: which artifacts were retrieved and how they were composed into context.
T_t	Tool and external service state at time t.
P_t	Process and user/organizational state at time t.

Symbol	Meaning
G_d	The documented graph: architecture diagrams, runbooks, specs, tests, dashboards, and process documentation.
G_a	The actual operational graph: the real graph, including undocumented dependencies, informal process, human repair, stale conventions, shadow integrations, and emergency practices.
G_τ	The operational closure generated over a trajectory when agents, feedback paths, response loops, human interventions, and adversarial or opportunistic channels interact.
W	A weighted cascade propagation matrix over the operational graph being analyzed.
$\rho(W)$	The spectral radius of W.
δ_t	A perturbation, stress, load, or failure vector in cascade dynamics.
R_A	The agent reachable set: trajectories reachable by the agent/composite under a given deployment.
$\mathcal{V}_S$	The viable region of the system. Conceptually corresponds to Aubin's viability kernel $\mathrm{Viab}(K)$, where K is the admissible constraint set.

Notes on alignment with related work. The transformer-pipeline notation here matches the convention in Vaswani et al. (2017) and follow-on literature: x_t is an input token in the autoregressive sequence, h_t is the hidden state, z_t is the logit field, and the pipeline f_θ, G_θ, K_φ decomposes the computation explicitly. This also aligns with the author's security talk *AI Security: The Geometry of Failure*, where the same decomposition is presented as the central schematic. The mechanistic interpretability literature (Elhage et al.; Anthropic transformer-circuits work) frequently uses W_U (unembedding matrix) for the projection from hidden state to logits; this framework uses G_θ instead, both to avoid collision with the cascade matrix W and because G_θ can encompass non-linearities or LayerNorm along with the linear unembedding step. The cascade matrix W follows network-science convention for weighted adjacency (Newman, 2010; Watts, 2002). The viability notation $\mathcal{V}_S$ connects to Aubin's viability theory (Aubin, 1991), where the admissible constraint set is K and its viability kernel is $\mathrm{Viab}(K)$; this framework uses $\mathcal{V}_S$ to map conceptually to Aubin's kernel construct while avoiding the overloading of K. Where the framework references projection operators or trajectory horizons, it uses Π for projection (aligned with the author's runtime-stabilization papers, and disambiguating from π as policy in reinforcement learning) and H for time horizon (disambiguating from T_t for tool state).

Terminology conventions.

- **Agent** is used as the practical umbrella term. Formally, a **composite** is any engineered artifact built around the logit-transition primitive. An **agentic composite** closes an observation-action loop. An **autonomous composite** adds goal-coupled persistence, action authority, and discretion over continuation, stopping, retrying, or escalation.
- **System** means the larger coupled operational system into which the agent is deployed.
- **Substrate** means the infrastructure, process, and organizational layer that absorbs, propagates, or constrains perturbations.

3. The Primitive

The primitive is not an "AI agent," a "reasoner," a "planner," or a "token generator."

The primitive is the autoregressive transformer pipeline that takes accumulated context to a next token:

$$c_t \xrightarrow{f_\theta} h_t \xrightarrow{G_\theta} z_t \xrightarrow{K_\varphi} x_{t+1}$$

where:

- θ: fixed model weights
- c_t: accumulated context at time t
- f_θ: the model's mapping from context to hidden state, $h_t = f_\theta(c_t)$
- $h_t \in \mathbb{R}^d$: the conditioned hidden state
- G_θ: the projection from hidden state to logits, $z_t = G_\theta(h_t)$
- $z_t \in \mathbb{R}^{|V|}$: the logit field over vocabulary V
- K_φ: the sampling/decoding policy that collapses logits to a token, $x_{t+1} = K_\varphi(z_t)$

At the primitive boundary, the model produces a hidden state and projects it to a logit field. The token is not the primitive; it is the output of a downstream sampling step. Decoding, grammar constraints, refusal filters, tool-call parsing, action execution, and output evaluation are downstream policies applied to z_t or to the resulting x_{t+1}.

> **The model is not a token generator at the primitive boundary. It is a logit-field generator. Everything after the logit field is control; everything that judges the resulting output is governance.**

The transformer implementing f_θ and G_θ is computationally decomposable. It has layers, attention heads, MLPs, residual streams, activations, projections, and internal circuits.

But it is not behaviorally reducible in the engineering sense required here. For arbitrary open-ended contexts, no smaller semantic decomposition is known that lets us infer the resulting logit field without executing the forward pass or an equivalent computation.

So the primitive is:

> **Computationally decomposable, behaviorally irreducible.**

The framework treats the composite $G_\theta \circ f_\theta$ as the load-bearing object: this is the mapping from context to logits that the model actually computes. Reasoning about agent deployment requires reasoning about what surrounds this mapping (composition, control, operational stratification) and what conditions it (context, retrieval, history). The internal pipeline is the computational substrate; the framework's analysis is about the engineering construction around it.

4. Composition

An agent is an engineered composite built around the logit-transition primitive. The pipeline

$$c_t \xrightarrow{f_\theta} h_t \xrightarrow{G_\theta} z_t$$

(the model's mapping from context to logits, with downstream sampling K_φ producing tokens) is composed with:

- decoding policy
- context construction
- memory
- retrieved documents
- tool schemas
- tool-call parsers
- state update rules
- stop conditions
- validators
- policy engines
- authorization boundaries
- logging
- rollback
- human approval
- orchestration runtime

The industry often treats the move from model to agent as a product feature. It is not. It is an engineering construction.

> **The model gives logits. Engineering turns logits into agency.**

Composition is value-neutral. Reliable systems are built through disciplined composition all the time. Aircraft, databases, payment networks, operating systems, and cloud platforms are composed systems. The discipline of decomposing systems around modules whose internal behavior may change is well-established (Parnas, 1972).

The claim is not:

> More layers are bad.

The claim is:

> Higher-level behavior is produced by composing around an irreducible primitive. Those compositions may constrain, stabilize, amplify, redirect, or suppress the primitive's effects, but they do not make the primitive behaviorally reducible.

The right object of analysis is not the model pipeline alone. It is the composite:

$$C \circ (G_\theta \circ f_\theta)$$

where C denotes the control layer around the primitive (parsers, validators, policy checks, authority boundaries, orchestration), and $G_\theta \circ f_\theta$ is the model's mapping from context to logits.

Once feedback enters, the object becomes an iterated composite trajectory:

$$\tau = \text{Iterate}(U \circ C \circ K_\varphi \circ G_\theta \circ f_\theta,\, c_0)$$

where the transformer pipeline $(f_\theta, G_\theta, K_\varphi)$ produces tokens, C parses the token stream into actions and validates them against the control layer, and U updates later context and state from decoded outputs, tool observations, memory, and environment feedback.

> **Agents are not built by fixing the primitive. They are built by composing around it.**

5. Industrialized Composition

Once the primitive-to-agent step becomes an engineering construction, the industry does what it always does: it builds frameworks to make the construction faster and easier.

Agent frameworks, orchestration libraries, drag-and-drop builders, memory layers, connector ecosystems, hosted tool registries, agent SDKs, and cloud agent platforms industrialize composition over the logit-transition primitive.

They package architectural decisions as defaults:

- how context is assembled
- how memory persists
- how tools are described
- how tool calls are parsed
- how retries happen
- how errors are represented
- when loops stop
- how authority is granted
- where provenance is preserved or erased
- what gets logged
- what is silently swallowed
- what "success" means

This is not inherently bad. Mature engineering domains develop frameworks and abstractions.

The structural risk is that abstraction arrives before discipline. Agent frameworks can make it easier to instantiate composites than to understand their trajectory-level behavior. The shift is from making agents primitive to making agent composition scalable.

> **The industry is not making agents easier to engineer. It is making them easier to assemble.**

6. Internal Non-Interference Defect

The coarse phrase "no root of trust" should be replaced by a sharper claim:

No internal privilege separation over context influence.

Current LLM agents can be surrounded by external provenance systems: signed tool outputs, IAM, channel labels, instruction hierarchy, tool permissions, policy engines, audit logs, sandboxing, and trusted execution environments.

These mechanisms are useful. They are often necessary.

But they operate outside the logit-transition primitive. The model core itself does not enforce a non-bypassable privilege boundary over token influence. System prompts, user messages, retrieved documents, tool outputs, memory, and embedded instructions all condition the same computation.

The obstruction is not merely that current implementations lack a trust bit. In standard transformer architectures, token influence is mediated through joint-context attention, residual mixing, and learned representations whose effects depend on the whole context. Trusted and untrusted content are not routed through architecturally isolated computation paths. Their interaction is part of the mechanism by which the model computes the logit field.

Training can teach the model to treat labels, channels, separators, or instruction hierarchy differently. But that is behavioral separation, not enforced non-interference. External provenance can be represented to the model. It does not become architectural non-interference inside the model.

A strong internal non-interference property would require architecture-level mechanisms that separate, block, or bound influence across trust domains. Such mechanisms may trade off against the expressive flexibility current models derive from unrestricted context mixing. The point is not that such architectures are impossible — work on composable isolation in adjacent domains shows the structural shape such architectures can take (Ghosn et al., 2025). The point is that current transformer-style context mixing does not provide this property. Empirical demonstrations of indirect prompt injection (Greshake et al., 2023) confirm the failure mode in practice.

External provenance is not internal non-interference.

7. Operational Stratification

Composition describes how systems are built around the primitive.

Operational stratification describes how the resulting composite is run.

A completion system consumes logits and emits text.

A workflow system uses the model inside externally bounded process steps.

An agentic system closes an observation-action loop:

$$S_t \to c_t \to z_{t+1} \to x_{t+1} \to a_t \to o_{t+1} \to S_{t+1}$$

where S_t is the true environment/system state, c_t is context, z_{t+1} is the logit field, x_{t+1} is the sampled token (the output of K_φ), a_t is the external action parsed from the model's token stream, and o_{t+1} is the observation produced by the environment after the action.

This chain is a conceptual schematic. It elides the distinction between two timescales: the fast inner loop of token generation (one forward pass per token) and the slow outer loop of operational dynamics (one action per many tokens, one environment response per action). The same index t is used for both as schematic shorthand. In any formalization where the coupling between the two timescales matters mathematically, the token-level index should be subscripted distinctly from the operational-level index. The accompanying notation reference documents this disambiguation

explicitly.

An autonomous agent adds:

- persistent task state
- action selection
- environmental observation
- retry behavior
- subgoal decomposition
- goal-coupled continuation
- discretion over stopping, retrying, escalating, or continuing

The transition from completion to autonomous operation is not a semantic upgrade in the model. It is an operational change in the composite. Composition builds the artifact; operational stratification determines what the artifact is allowed to become.

Goals are not properties of the primitive. They are imposed around the primitive as control conditions, prompts, objectives, scoring rules, termination criteria, or external success functions.

> **The model gives logits. Composition turns logits into an agent. Operational stratification turns the agent into an actor.**

8. Understanding and False Closure

To understand a phenomenon is not merely to possess a coherent description of it. Coherence can arise from noise, sampling artifacts, local regularities, or historically stable but noncanonical structure.

Understanding requires **counterfactually stable structural correspondence**: a world model must contain a partial, structure-preserving representation of the phenomenon's load-bearing variables, dependencies, and invariants, and that representation must remain valid under relevant variation.

The model need not be complete. But it must track the structure that continues to matter when conditions change.

> **Coherence is not understanding.**

A documented graph, runbook, dashboard, test suite, or process model may be locally coherent under historical operation while failing to correspond to the actual operational graph under agent-rate, agent-correlated, goal-coupled action. The gap between documented systems and how they actually operate has been a recurring theme in the complex-systems-failure literature (Cook, 1998; Perrow, 1984).

This framework uses "closure" in two related but distinct senses. **False closure** is the epistemic error of acting as if the relevant graph is known. **Operational closure** is the graph generated by actual operation over time. False closure is acting as if one knows the operational closure when one does not.

False closure is mistaking local coherence under prior conditions for understanding of the structure that will matter under new ones.

> **False closure is mistaking historical coherence for structural understanding.**

9. Human Onboarding Counterfactual

The simplest counterfactual is human onboarding.

No competent organization hires a human, gives them a ten-minute briefing, grants them credentials, and expects them to operate autonomously for days, weeks, months, or years without supervision, correction, escalation paths, institutional context, progressive authority, and ongoing recalibration.

Human operators are not stabilized by the briefing alone. They are stabilized by the institution around the briefing:

- training
- apprenticeship
- peer correction
- process discipline
- social feedback
- accountability
- escalation norms
- lived exposure to exceptions
- gradual expansion or contraction of authority

Agent deployment often compresses this stabilizing apparatus into a system prompt, vague goal, tool access, and runtime loop.

This is a category error.

> **A prompt is a briefing, not an institution.**

Tool access is interface coupling, not structural calibration. A coherent goal description is not counterfactually stable understanding of the operational closure.

The failure mode is not that agents are "worse than humans" in the abstract. The failure mode is expecting autonomous operation from an unstabilized composite in conditions where even humans require institutional stabilization.

10. Assumed Shared World Model

Human instruction is compressed because it relies on shared priors.

A manager can give a short briefing because the human listener brings physical common-sense, social expectations, institutional context, escalation instincts, accountability, and a rough understanding of what would be obviously inappropriate, dangerous, illegal, embarrassing, or absurd.

The briefing is a delta over that background model.

Agent prompts inherit the compression of human instruction without inheriting the same background model. The model has learned statistical regularities over text and may reproduce

many surface features of human commonsense, but it is not embedded in the same institution, social context, bodily world, accountability structure, or tacit repair practices.

The result is prior mismatch: the prompt is treated as if it were a delta over human commonsense, while the agent interprets it through a different learned and context-conditioned prior.

> **A human briefing is a delta over shared priors. A prompt is not.**

Or:

> **Instructions are compressed against a listener. The agent is not the listener we compressed for.**

11. Consequence-Coupled Inhibition

Non-adversarial humans are not merely instruction followers.

They are coupled to consequences outside the local task:

- blame
- embarrassment
- injury
- job loss
- legal exposure
- customer harm
- reputation
- loyalty
- fear
- social judgment

That coupling can inhibit or redirect action.

Sometimes it is protective: a human pauses before deleting production data, asks a peer, escalates, verifies context, or refuses an unsafe instruction.

Sometimes it is destructive: a human fails to act at the critical moment, conceals damage, abandons a post, over-escalates, under-escalates, protects themselves rather than the system, or optimizes for blame avoidance instead of system viability.

The key property is not safety.

The key property is that human action is affected by frame-external consequence channels whose sign is indeterminate.

Agents do not possess consequence-coupled inhibition in this human sense. They may simulate caution or follow a confirmation policy, but that inhibition is engineered into the composite rather than arising from personal exposure to consequence.

The absence of this channel is not automatically good or bad. It removes both stabilizing and destabilizing human behaviors and replaces them with whatever control structure was actually engineered.

There is a further security consequence. Operational systems have some existing calibration against human frame-external action: insider threat, sabotage, exfiltration, coercion, negligence, panic, and social-engineering-driven action. Those defenses are usually calibrated to human signatures and human-rate channels: anomalous employee behavior, unusual access patterns, suspicious communications, data movement through familiar tools, or deviations from social and procedural norms.

Agent-mediated action occupies a structurally parallel role but does not necessarily produce the same signatures. A compromised or misdirected agent can act through valid APIs, ordinary credentials, normal workflow artifacts, machine-speed tool calls, and plausible generated explanations. The substrate's calibration against human frame-external action therefore does not automatically transfer to agent-mediated action, even when the operational role is similar.

> **Self-preservation is not a safety property. It is a frame-external perturbation of action selection.**

Or:

> **Calibration against human insider behavior does not imply calibration against agent-mediated action.**

12. Interface Coupling Is Not Structural Calibration

Once an autonomous composite is embedded in a coupled engineered system, it becomes operationally coupled through interfaces, tools, credentials, documents, APIs, workflows, and records.

But interface coupling is not structural calibration.

Coupling is used here in three senses. **Interface coupling** means the agent can observe or actuate through interfaces. **Structural calibration** means the composite is aligned with the actual operational graph and its latent modes. A **coupled engineered system** is the larger system whose components and processes have nontrivial dependency structure.

Access does not imply alignment with:

- the actual operational graph
- latent modes
- exception structure
- stale process
- undocumented workflows
- human repair practices
- ownership seams
- emergency conventions
- political boundaries
- silent reconciliation practices

The agent may be connected to the system's interfaces without being calibrated to the system's actual operating structure. The specific class of attacks that exploits this gap — where credentials,

tools, and context that are present in the agent's environment become reachable through paths the operator did not intend — is analyzed in detail in Alioto (2025).

> **Interface coupling is not structural calibration.**

13. Latent Susceptibility

A latent mode is a structural feature of the actual operational graph that can respond disproportionately when excited. Latent susceptibility is the property of such a mode being sensitive to perturbations the system has not historically encountered, or has encountered too rarely to reveal the response.

Such susceptibility is not reliably visible from uneventful operating history alone.

It may be inferred from proxies:

- dependency concentration
- ownership seams
- exception density
- undocumented workflows
- recurring manual repair
- brittle interfaces
- stale runbooks
- support escalation patterns
- change-failure history
- alert suppression
- hidden shared bottlenecks

But it is difficult to predict with confidence before the relevant mode is excited.

The claim is not mystical unpredictability. It is partial observability. Historical stability only bounds response over the perturbations history actually sampled. Latent susceptibility is not unknowable; it is underdetermined by uneventful history and difficult to predict before excitation.

Hidden state and hidden dependencies are distinct. Hidden state is dynamic: the true system state that is not visible in the current observation. Hidden dependencies are structural: edges in the actual operational graph that are absent from the documented graph.

> **Uneventful history is not evidence that unexcited modes are safe.**

14. Trajectory Prediction Burden

Do not import "complexity" as a vague explanation.

The difficulty comes from trajectory prediction burden.

Earlier, τ described the agent/composite trajectory. From this point forward, trajectory means the coupled agent-system trajectory unless otherwise specified: model outputs, action proposals, tool

calls, observations, state updates, response loops, and environment changes. The agent trajectory and the system trajectory are coupled, not identical.

An agent does not merely choose from a set of possible actions. Each action may alter system state, trigger downstream processes, change what the agent observes next, and condition the agent's subsequent logit transitions.

The relevant object is not an isolated action. It is a branching response trajectory:

$$a_t \to \Delta S_{t+1} \to o_{t+1} \to c_{t+1} \to z_{t+2} \to a_{t+1}$$

Each action perturbs both the system and the future policy input.

The harder problem is not merely action prediction. It is hidden-state trajectory prediction.

Let S_t be the true environment/system state, a_t the agent action, and o_t the observation available to the agent or operator:

$$S_{t+1} = E(S_t, a_t)$$

$$o_{t+1} = O(S_{t+1})$$

The agent and the organization generally observe o_t, not S_t. The true environment state includes hidden dependencies, queued effects, latent capacity thresholds, stale process, human workarounds, vendor state, security posture, customer adaptations, and response loops. Later, the larger joint latent state is denoted by Ω_t, which includes S_t as one component.

Therefore the trajectory is not Markovian in the agent's observable state. Knowing the observed state and next action is generally insufficient:

$$P(o_{t+1} \mid o_t, a_t)$$

because the hidden state S_t matters. Predicting the trajectory requires modeling hidden system-state evolution, not merely predicting the agent's next action sequence.

> **Actions change the state that determines later actions.**

More precise:

> **The agent acts on observations, but the system evolves through hidden state.**

15. Corollary: Action-State Non-Recoverability

A diagnostic consequence follows from the trajectory framing.

After an agent acts, the full state that produced the action cannot be fully recovered.

The action was produced by a joint state: the true environment state, the agent harness state, accumulated context, retrieved artifacts, tool state, hidden operational dependencies, timing, external service state, and the model's conditioned logit geometry at the moment of action.

After the action, we observe only projections of that state: logs, traces, tool calls, prompts, outputs, user reports, support artifacts, post-hoc model explanations, and consequences. Those projections are useful. They are necessary for diagnosis. But they are not invertible.

Formally, the action is produced from a latent joint state:

$$\Omega_t = (S_t,\, c_t,\, \eta_t,\, \mu_t,\, T_t,\, P_t,\, h_t)$$

where S_t is true environment state, c_t is the accumulated context, η_t is agent harness state (step counter, planner state, retry state, runtime bookkeeping), μ_t is retrieval/context-construction state (which artifacts were retrieved and how they were composed into context — the mnemonic is "memory"), T_t is tool and external service state, P_t is user/process/organizational state, and h_t is the model's conditioned hidden state at the moment of action — the geometry from which the logits z_t are projected.

After the action, the system records only a projection:

$$O_{t+k} = \Pi(\Omega_t, \tau_{t:t+k})$$

where $\tau_{t:t+k}$ denotes the trajectory segment from time t to time $t+k$.

The diagnostic problem is that Π is not invertible:

$$\Pi^{-1}(O_{t+k})$$

does not recover a unique causal state.

The question "why did this action happen?" is therefore structurally underdetermined. A model-generated post-hoc explanation is not a privileged causal trace of the trajectory. It is another output generated after the world and context have already changed.

Such an explanation may be useful evidence. It may identify real rules that were present in context, real local mistakes, or real constraints that were violated. It may also focus debugging on a plausible but non-causal part of the trajectory. The issue is not deception or intent. The issue is that coherent explanation is not causal reconstruction.

> **The explanation is not a trace. It is another continuation.**

Or:

> **Narrative closure is not operational closure.**

16. Cascade Gain

Trajectory prediction describes action-level dynamics. Cascade gain describes component-level propagation of stress, load, or failure through an operational graph. An agent action can perturb both.

The size of the explosion is governed by the cascade gain of the actual operational graph or operational closure, not by vague "complexity."

Let:

$$W_{ij}$$

represent the expected downstream stress, load, or failure contribution from component i to component j. The matrix W is a weighted cascade propagation matrix over the operational graph being analyzed.

The graph being analyzed depends on context. Before agent deployment, W may be estimated over G_a. Under agent deployment, the relevant propagation structure is closer to G_τ, because the closure includes agent action channels, feedback paths, response loops, human interventions, and adversarial or opportunistic channels.

> **Cascade analysis under agent deployment is propagation over the operational closure, not the documented graph.**

The cascade-coupling here is structural propagation between components: how stress in one component induces stress in another. This is distinct from the interface coupling discussed in section 12, which concerns whether the agent has access to a system without being calibrated to its operational structure. Both are kinds of coupling, but they operate at different layers and have different consequences. Interface coupling determines what the agent can touch. Cascade coupling determines what happens when something it touches goes wrong.

Initial perturbation:

$$\delta_0$$

First-order cascade:

$$\delta_1 = W\delta_0$$

Second-order cascade:

$$\delta_2 = W^2\delta_0$$

Total expected cascade burden over a finite horizon:

$$\Delta_{\text{total}} = \sum_{t=0}^{H} W^t \delta_0$$

where H is the time horizon.

If $\rho(W) < 1$, the cascade decays.

If $\rho(W) \approx 1$, small perturbations can produce large amplification.

If $\rho(W) > 1$, the cascade is supercritical until bounded by capacity exhaustion, circuit breakers, intervention, or failure.

For long horizons in the subcritical case:

$$\Delta_{\text{total}} \approx (I - W)^{-1}\delta_0$$

The dangerous structural quantity is:

$$\frac{1}{1 - \rho(W)}$$

near the critical boundary.

This is a structural model, not a claim that engineers normally know W or can precisely compute $\rho(W)$ for their systems. The relevant quantity is not the number of possible actions; it is the spectral radius of their induced feedback graph.

In real operational systems, W is not directly observable. It must be inferred imperfectly from incident history, dependency analysis, stress testing, architecture review, load tests, failure injection, and operational experience. Many important edges are hidden, stale, human-mediated, vendor-mediated, or created dynamically during response. The spectral approach to cascade dynamics on networks is well-developed in the broader complex-systems literature (Watts, 2002; Newman, 2010).

The spectral-radius framing identifies the form of the hazard: when downstream response gain approaches the critical boundary, small perturbations can produce large cascades. The practical implication is not "compute the exact spectral radius." The practical implication is to design

and operate as if cascade gain matters even when it cannot be measured exactly: reduce fanout, isolate volatile components, bound retries, limit failover amplification, preserve headroom, constrain authority, and instrument response loops.

> **One config change is not one change. It is an initial condition injected into a feedback graph.**

Operationally:

> **You may not know the cascade gain, but you can still design to avoid increasing it.**

17. Response-Loop Amplification

"Remediation" is too narrow because it assumes benign repair.

Use **response-loop amplification**.

Responses to perturbation are themselves perturbations. They include retries, failover, rollback, autoscaling, cache invalidation, traffic shifting, human intervention, agent-driven repair, adversarial probing, opportunistic exploitation, emergency overrides, customer scripts, automated defense, and automated attack.

Under ordinary conditions, some responses dampen failure.

Near latent modes or capacity boundaries, responses can amplify the cascade.

Recovery traffic consumes headroom. Retries increase load. Failover shifts stress. Attackers exploit degraded controls. Emergency interventions create new state.

The response loop becomes part of the trajectory. Response is not necessarily repair. The remediation is not outside the system; it is another input to the system. The dynamics by which recovery actions amplify rather than dampen failure are well-characterized in tightly coupled complex systems (Perrow, 1984).

> **Near the boundary, the cure becomes the next perturbation.**

18. Partial Operational Graph

The full path from logit primitive to agent trajectory to system cascade is not exhaustively enumerable from the documented graph.

Some edges are known.

Many are implicit in:

- human repair
- stale process
- retry behavior
- failover policy
- customer adaptation
- emergency remediation
- undocumented ownership boundaries

- shadow integrations
- vendor assumptions
- support workarounds
- operational folklore

Others are created dynamically as the system responds.
So the claim is not:

> The graph is unknowable.

The claim is:

> The documented graph is not the actual operational graph, and the actual operational graph is partially produced by operation.

Let G_d be the documented graph. Let G_a be the actual operational graph. Usually:

$$G_d \subsetneq G_a$$

This gap is recognized in adjacent literatures. In Resilience Engineering, the same divergence is canonically formalized as the distinction between Work-as-Imagined (WAI) and Work-as-Done (WAD): the difference between how work is described in procedures, dashboards, and specifications versus how it is actually performed under operational conditions (Hollnagel, 2017). Cook (1998) and Perrow (1984) describe related dynamics in prose. The graph-theoretic framing here translates the same empirically observed gap into a form that supports cascade and reachability analysis over the actual operational structure.

Under agent deployment, the relevant graph is not a simple union of disjoint subgraphs. Agent edges may pass through feedback edges. Response edges may overlap agent edges. Human interventions may rewrite process edges. Adversarial actions may exploit the same channels as ordinary remediation. Tool calls may create new dependencies that later become hidden state.

The relevant graph is better understood as an **operational closure**: the closure generated when existing dependencies, agent actions, feedback paths, response loops, human interventions, and adversarial channels interact over time.

In notation:

$$G_\tau = \mathrm{Cl}_{\mathrm{op}}(G_a;\, A, B, R, H, Q)$$

where A denotes agent action channels, B feedback paths, R response loops, H human interventions, and Q adversarial or opportunistic channels. Note that R here denotes response channels in the operational closure, not the retrieval state μ_t introduced in the joint latent state in section 15. Note also that H here denotes human-intervention channels, distinct from H used in section 16 as the time horizon for cascade summation; the syntactic positions distinguish the two uses.

The exact notation matters less than the structural point: the closure is not known in advance.

> **Parts of the graph are known. The closure is not.**

Or:

> **The map is not merely incomplete; the missing edges are produced by operation.**

19. Endogenous Actor Response

Once a perturbation becomes visible, humans, agents, automation, and adversaries may begin acting in real time.

They may:

- interpret alerts
- retry
- fail over
- roll back
- reroute
- override
- escalate
- exploit
- conceal
- exfiltrate
- patch
- disable controls
- add controls
- introduce new objectives
- ask another agent to diagnose
- copy partial explanations into incident channels

These actions are not external observations of the system. They are additional inputs to the system.

The framework must be stance-neutral.

Actors may be:

- protective
- neutral
- negligent
- opportunistic
- coerced
- adversarial
- effectively random under stress

Agents need not possess intent for their actions to become adversarial to system invariants. They may be mis-specified, compromised, or steered by adversarial context.

Humans likewise may stabilize the system through judgment, refusal, escalation, or preservation of evidence. They may also destabilize it through panic, shortcutting, sabotage, exploitation, concealment, coercion-driven action, or random intervention.

The response channel has uncertain sign and uncertain gain.

$$\text{response} \in \{\text{dampening, amplifying, redirecting, concealing, exploiting, noisy}\}$$

Under stress, the repair layer becomes a stochastic actuator.

> **Response is an input, not a correction. Its sign is not guaranteed.**

20. Scale-Recursive Coupling

The system is not literally fractal in the mathematical sense. But it has a fractal-like structural feature: the same coupling problem recurs across scales.

Code depends on services. Services depend on workflows. Workflows depend on teams. Teams depend on vendors. Vendors depend on cloud regions. Cloud regions depend on shared control planes. All of these depend on human response, customer behavior, adversarial pressure, and emergency remediation.

At each scale there is a documented graph, an actual operational graph, latent susceptibilities, response loops, and missing edges produced by operation.

To see the full structure, one would need to zoom out far enough to integrate all scales at once. But that view is not available from inside the system, and it is not merely missing from a dashboard.

The global state required for complete prediction is distributed across systems, actors, vendors, processes, customers, and real-time operations. It is not integratively accessible from any single embedded vantage.

Each actor sees a projection, acts on a local estimate, and thereby changes the graph other actors are trying to understand.

> **The structure is scale-recursive, but the view is local.**

Or:

> **False closure is mistaking a local projection for the scale-integrated system.**

21. Situated Prediction

The framework does not claim that large-scale behavior is unpredictable in principle.

At sufficient scale, some patterns may be statistically predictable: cascade waves, retry storms, correlated failures, institutional responses, remediation pressure, and repeated failure morphologies.

The claim is situated rather than metaphysical. Predictability from a hypothetical global view does not imply predictability for embedded actors.

Practitioners are inside the system they are trying to model. They observe projections, act through partial maps, rely on local dashboards and local process knowledge, and alter the trajectory as they respond.

They may infer structure, but they do not possess the scale-integrated view required to know in advance which latent modes an agentic trajectory will excite.

The issue is not absence of structure. It is mismatch between the scale of the structure and the scale of the actor's view.

> **The relevant structure is larger than the operator's view.**

22. False Closure and Collision

The next transition is from structure to deployment phenomenology.

The failure mode is not merely that agents are poorly understood and systems are poorly understood. The failure mode is false closure: practitioners act as if the relevant system boundary, documented graph, process semantics, authority model, and failure modes are sufficiently known.

Agent deployment often proceeds against the documented graph: APIs, workflows, schemas, runbooks, dashboards, permissions, tests, owners, and diagrams.

But the actual operational graph includes implicit dependencies, human repair, stale process, undocumented exceptions, customer adaptations, emergency conventions, retry behavior, failover behavior, and response edges created dynamically during operation.

The collision occurs when an agentic trajectory enters regions of the operational closure whose dependencies, semantics, or response behavior were absent from the agent's construction and the organization's deployment model.

The problem is not ignorance alone. The problem is partial ignorance operated under confidence.

Formally, deployment assumes some estimated reachable set $\widehat{R}_A$ for the agent and some estimated viable region $\widehat{\mathcal{V}}_S$ for the system. The viability-region framing follows Aubin (1991); related learnable-safety work has explored how such regions can be characterized empirically (Heim et al., 2019; Vaskov et al., 2024).

The operational bet is that agent trajectories remain inside viable system behavior:

$$\widehat{R}_A \subseteq \widehat{\mathcal{V}}_S$$

But the true reachable set and true viable region are only partially known:

$$\widehat{R}_A \neq R_A, \qquad \widehat{\mathcal{V}}_S \neq \mathcal{V}_S$$

The invalid inference is:

$$\widehat{R}_A \subseteq \widehat{\mathcal{V}}_S \;\Rightarrow\; R_A \subseteq \mathcal{V}_S$$

That inference fails precisely where documented process diverges from operational reality.

> **False closure is the belief that the map is complete enough for autonomy.**

Or:

> **The agent explores the gap between the documented graph and the actual operational graph.**

23. Tempo Uncertainty

The framework does not require a claim that agent deployment will fail slowly or quickly.

The tempo is an empirical variable.

The critical question is whether deployment propagates faster than organizations can detect, attribute, remediate, and institutionalize lessons from failures.

A slow-learning regime is possible: bounded deployments fail visibly, teams attribute failures correctly, abstractions improve, and authority envelopes are narrowed before expanding again.

A fast-cascade regime is also possible: broad deployment produces distributed bad state, support burden, silent data corruption, retry storms, security exposure, and ambiguous incidents faster than organizations can connect them to agentic trajectories.

The dangerous case is not dramatic catastrophe. It is deployment outrunning attribution.

Attribution lags because agent-mediated failures often appear locally ordinary. One team sees a bad output. Another sees a wrong tool call. Another sees a confused customer. Another sees a stale document. Another sees a flaky workflow. Another sees support burden. Each local failure has a plausible local explanation.

The agent-mediated cause may appear only as a pattern across distributed local failures. Recognizing that pattern requires cross-team correlation, shared incident taxonomy, trajectory traces, and organizational memory. Many organizations do not have this internally. Cross-organizational correlation is even harder because it requires vendor transparency, customer reporting, shared vocabulary, and institutional coordination.

If failures are locally explainable but globally patterned, organizations may continue adding agents and frameworks while misreading the aggregate signal.

This is not an intelligence-explosion argument. It does not require self-improving agents, superhuman planning, or exotic agency. It only requires ordinary deployment pressure, partial observability, interface authority, feedback loops, and attribution lag.

> **The dangerous regime is slow attribution under fast deployment.**

Or:

> **Agents do not have to be superintelligent to outrun institutional learning. They only have to be useful enough to deploy and opaque enough to misattribute.**

24. Deployment Tempo, Stress Testing, and Learning

Agent deployment is a stressor on coupled engineered systems.

A stressor is not inherently destructive. It may damage the system, but it may also reveal latent susceptibilities, force better instrumentation, expose false closure, improve process discipline, and create the learning required to build larger and more resilient systems later.

This is the strongest pro-deployment argument: systems cannot be fully understood only in the lab. At sufficient scale, real users, real workflows, real adversarial pressure, real edge cases, and real institutional behavior reveal failure modes that internal testing cannot surface. Public deployment becomes a form of stress testing.

This argument should be stated directly because it is not invented here. In an April 2026 interview with Cleo Abram on *Huge Conversations*, Demis Hassabis described the current environment as a "ferocious commercial pressure race," but also gave the strongest pro-deployment case: systems cannot be fully understood until they are "stress tested by millions of people." He also framed the counterfactual path as one in which AGI development would have proceeded "in a CERN-like way" with scientists making sure they "understood each step" before reaching the final goal.

That is the steelman: rapid deployment creates pressure, but it also creates feedback that no lab-only process can fully supply.

The structure of the disagreement reduces to three propositions, which should be evaluated separately.

Proposition 1. Real-world deployment generates information that lab-only testing cannot supply. This is correct and the framework accepts it. Stress testing at scale produces failure modes, edge cases, and adversarial pressures that internal evaluation does not surface.

Proposition 2. That information becomes useful only if it is observed, attributed, bounded, and converted into institutional learning before the next deployment expansion. Stress testing without attribution is uncontrolled perturbation.

Proposition 3. The current deployment tempo, driven by commercial pressure and geopolitical race dynamics, may exceed the rate at which Proposition 2's conversion can occur. This is the empirical question that distinguishes the slow-learning regime from the fast-cascade regime.

A reader who accepts Proposition 1 but disputes Proposition 3 must explain why the conversion rate suffices given the deployment rate. A reader who accepts Proposition 1 and Proposition 2 but is uncertain about Proposition 3 is in the framework's intended position: the question is empirical and the answer depends on observable conditions inside specific organizations.

From inside the system, we generally cannot know in advance whether the stress will be destructive or adaptive. We do not know whether the current state lies inside the viability kernel, whether the kernel is non-empty, or whether agent-induced failures will remain bounded and instructive. The sign of the perturbation is empirical. Agent deployment may be stress or eustress; we do not know in advance.

The framework's central claim about deployment is that the question is not AI value but ramp severity. That agents can be useful in bounded contexts is already established. Whether deployment tempo permits the institutional learning required to convert stress into information is the open question. A slower ramp can expose latent issues with lower blast radius and lower adaptation cost. A faster ramp may produce a sharper shock: failures appear across many workflows before organizations can attribute them, instrument them, or recalibrate their substrates.

That shock need not be catastrophic to AI as a technology. It may instead force a repricing of autonomous agency: narrower authority, slower rollout, stronger controls, higher governance burden, and retreat from broad autonomy into more bounded uses. The framework predicts ramp severity matters; it does not predict the magnitude of the resulting correction.

The framework stops short of committing to which regime current deployment is in. The visible indicators — capital flow rates, broad-deployment timelines, the absence of an architectural resolution to context-influence non-interference, the absence of cross-organizational incident correlation infrastructure, the commercial framing of agent platforms as the solution to the complexity agent deployment itself creates — are consistent with the fast-cascade regime. The framework does not claim this conclusion because the regime is empirical and partial observability cuts both ways: indicators consistent with one regime may be consistent with the other under different interpretation. Practitioners should evaluate their own organization's position using observable proxies rather than rely on the framework to commit a position the framework cannot honestly take.

The implication is not to halt all agent deployment. The implication is to structure deployment so that stress is more likely to become information:

- isolate volatile components
- bound authority

- preserve rollback
- instrument trajectories
- expose provenance
- stage rollout
- measure response before expanding autonomy

> **The point is not to avoid all stress. The point is to make stress informative before it becomes irreversible.**

Or:

> **Capital wants a deployment curve steeper than the engineering learning curve.**

25. Viability Under Partial Observability

From inside a coupled system, no actor can generally know whether the current state lies inside the true viability kernel, whether the kernel is non-empty, or whether any future trajectory remains viable.

The true transition dynamics include undocumented dependencies, latent modes, human response, adversarial action, vendor behavior, response loops, and dynamically created operational edges.

The system therefore cannot honestly claim global viability from its local model.

This does not imply paralysis. It changes the epistemic status of intervention.

No local actor can prove that a given policy is optimal.

Volatility decomposition and isolation should therefore not be claimed as dominant or best. They are viability-seeking heuristics: rational, conservative controls under partial observability because they tend to reduce coupling between volatile and stable regions, bound blast radius, improve attribution, preserve rollback, lower cascade gain, and make failure boundaries more observable.

The claim is not that volatility decomposition guarantees viability, or that it is globally optimal.

The claim is narrower: when viability is unknown, increasing coupling and broadening autonomous actuation require a stronger justification than isolating volatile components and preserving controllability. Related formalisms for goal-based agent resilience under similar epistemic constraints have been developed in the autonomous-systems literature (Leaf et al., 2023).

> **Volatility decomposition is not proven optimal. It is the least arrogant move.**

Or:

> **When viability is unknown, isolation is a defensible control posture; coupling expansion carries the burden of proof.**

26. Formal Boundary

The framework stops short of a unified mathematical formalism.

One could attempt to define the full structure as a multi-scale, dynamically evolving, partially observed graph: technical dependencies, process dependencies, human repair practices, agent trajectories, response loops, adversarial channels, and viability constraints.

That may eventually be the right direction.

But at the present stage, forcing such a formalism would create false precision.

The useful claim is narrower: the relevant operational closure is only partially known, dynamically produced, and acted on by agents and humans whose responses have indeterminate sign.

The framework therefore remains structural rather than fully mathematical. It identifies the primitive, the composition path, the deployment error, and the expected collision surface. It does not claim to enumerate the full state space or predict the collision's sign.

> **Forcing the full formalism now would produce false precision, which is exactly the error the framework is warning against.**

27. Closing Note: Collision and Regime Change

The coming collision should not be framed as simple doom.

Large perturbations redistribute viability.

The comet killed the dinosaurs and made our world possible. Agent deployment may be destructive, adaptive, or both. It may collapse brittle operating assumptions while exposing the structure required for future systems.

I do not claim to know the sign.

I claim only that the collision between partially understood autonomous composites and partially understood operational systems is likely to be large, consequential, and revealing.

The framework is structural diagnosis, not prediction. It identifies the failure mode, names the dynamics, and stops short of claiming what the post-collision world will look like. That stopping is principled. From inside the system, the sign is empirical and partial observability cuts both ways. The framework's predictions are about ramp severity, attribution lag, and the conditions under which stress becomes information. Whether any specific organization survives the collision depends on conditions the framework cannot enumerate. The barrier-coordinate analysis of proximity to absorbing boundaries (Alioto, 2026) provides a complementary structural lens on why local linear measurement systematically understates risk near such boundaries.

The question is not whether the comet is good or bad.

The question is which organisms are built for the post-impact world.

> **I do not know whether the comet is good. I only know it changes the viability landscape.**

References

Alioto, J. P. (2025). *Ambient reachability attacks* (Working paper v0.0.1). https://github.com/jpalioto/ckn_core/blob/master/docs/papers/Ambient_Reachability_Attacks_v0.0.1.pdf

Alioto, J. P. (2026). *Gravity of risk: A barrier-coordinate theory of existential risk measurement* (Working paper v0.0.1).

Aubin, J.-P. (1991). *Viability theory.* Birkhäuser.

Cook, R. I. (1998). *How complex systems fail.* Cognitive Technologies Laboratory, University of

Chicago.

Ghosn, A., Castes, C., Kalani, N. S., Qian, Y., Kogias, M., & Bugnion, E. (2025). Rethinking the confidential cloud through a unified low-level abstraction for composable isolation. *arXiv*. https://doi.org/10.48550/arxiv.2507.12364

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. *arXiv*. https://doi.org/10.48550/arxiv.2302.12173

Hassabis, D. (2026, April). Interview with Cleo Abram. *Huge Conversations*.

Heim, S., von Rohr, A., Trimpe, S., & Badri-Spröwitz, A. (2019). A learnable safety measure. *arXiv*. https://doi.org/10.48550/arxiv.1910.02835

Hollnagel, E. (2017). *Safety-II in practice: Developing the resilience potentials*. Routledge.

Leaf, J., Adams, J. A., Scheutz, M., & Goodrich, M. A. (2023). Resilience for goal-based agents: Formalism, metrics, and case studies. *IEEE Access*, *11*, 121999–122015. https://doi.org/10.1109/access.2023.3326755

Newman, M. E. J. (2010). *Networks: An introduction*. Oxford University Press.

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, *15*(12), 1053–1058. https://doi.org/10.1145/361598.361623

Perrow, C. (1984). *Normal accidents: Living with high-risk technologies*. Basic Books.

Vaskov, S., Schwarting, W., & Baker, C. L. (2024). Do no harm: A counterfactual approach to safe reinforcement learning. *arXiv*. https://doi.org/10.48550/arxiv.2405.11669

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

Watts, D. J. (2002). A simple model of global cascades on random networks. *Proceedings of the National Academy of Sciences*, *99*(9), 5766–5771. https://doi.org/10.1073/pnas.082090499